

Java Backend Architecture

Interview Series

Chapter 8.10

RBAC, ABAC & Zero Trust

50+ Interview Q&A

Code Examples

Architecture Diagrams

Advanced Topics

Chapter 8.10 — RBAC, ABAC & Zero Trust

Introduction

Authorization models determine who can do what in your system. RBAC (Role-Based Access Control) is simple and scalable but coarse-grained. ABAC (Attribute-Based Access Control) is flexible and fine-grained but complex. Zero Trust is an architectural philosophy — no implicit trust based on network location; every request is authenticated and authorized. Together, these form the foundation of enterprise-grade authorization.

RBAC vs ABAC

RBAC: Role-Based Access Control

User → Role (ADMIN)

ROLE: PERMISSIONS (DELETE_USER)

ADMIN: READ+WRITE+DELETE

MANAGER: READ+WRITE

VIEWER: READ only

ABAC: Attribute-Based Access Control

Subject attrs: role, dept, clearance

Resource attrs: role, dept, clearance

Environment attrs: time, location, device

PERMIT if:

subject.dept == resource.dept

AND subject.clearance >= resource.level

AND time.hour BETWEEN 9 AND 17

RBAC: simple, scalable, coarse-grained. ABAC: flexible, fine-grained, complex. Combine for most enterprise systems.

Spring: @PreAuthorize("hasRole('ADMIN')") = RBAC. @PreAuthorize("@policy.check(#id)") = ABAC.

Dimension	RBAC	ABAC
Control Unit	Roles → permissions	Policies using attributes
Granularity	Coarse (role-level)	Fine (field/context-level)
Complexity	Low — easy to understand	High — complex policy management
Scalability	Good (role count can explode)	Better (policies independent of user count)
Dynamic Context	Limited (role is static)	Native (time, location, device)
Standards	NIST RBAC standard	XACML, ALFA, OPA Rego
Spring Support	hasRole(), @Secured	@PreAuthorize with SpEL, custom evaluators
Best For	Internal apps, simple permissions	Multi-tenant, data-level, compliance

RBAC Implementation in Spring

```
// RBAC with Spring Security
@Entity public class User { @ManyToMany(fetch = FetchType.EAGER)
private Set<Role> roles; }
@Entity public class Role { private String name; // ROLE_ADMIN,
ROLE_MANAGER, ROLE_VIEWER
@ManyToMany private Set<Permission> permissions; }
// Load authorities from roles
public Collection<GrantedAuthority> getAuthorities() { return
roles.stream().flatMap(role -> role.getPermissions().stream()).map(perm -> new
SimpleGrantedAuthority(perm.getName())).collect(Collectors.toList()); }
// Usage in controllers
@GetMapping("/admin/users") @PreAuthorize("hasRole('ADMIN')") public List<User>
getAllUsers() { ... }
@DeleteMapping("/users/{id}")
@PreAuthorize("hasAuthority('DELETE_USER')") public void deleteUser(@PathVariable Long id) {
... }
```

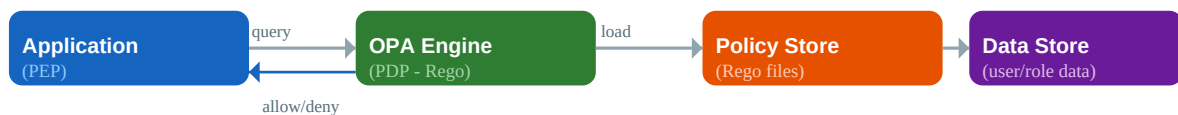
ABAC Implementation in Spring

```
// ABAC with Spring Security – custom permission evaluator
@Component public class OrderPermissionEvaluator implements PermissionEvaluator {
    private final OrderRepository orderRepository;
    @Override public boolean hasPermission(Authentication auth, Object targetDomainObject, Object permission) {
        if (targetDomainObject instanceof Order order) {
            User user = (User) auth.getPrincipal();
            return switch (permission.toString()) {
                case "READ" -> order.getUserId().equals(user.getId()) || user.hasRole("ADMIN");
                case "APPROVE" -> user.hasRole("MANAGER") && order.getAmount().compareTo(user.getApprovalLimit()) <= 0;
                default -> false;
            };
        }
        return false;
    }
}

// Register as MethodSecurityExpressionHandler
@Bean public MethodSecurityExpressionHandler expressionHandler( OrderPermissionEvaluator evaluator) {
    DefaultMethodSecurityExpressionHandler handler = new DefaultMethodSecurityExpressionHandler();
    handler.setPermissionEvaluator(evaluator);
    return handler;
}

// Usage
@PreAuthorize("hasPermission(#order, 'APPROVE')") public Order approveOrder(Order order) { ... }
}
```

Open Policy Agent (OPA)



PEP=Policy Enforcement Point. PDP=Policy Decision Point. OPA decouples policy from application code.

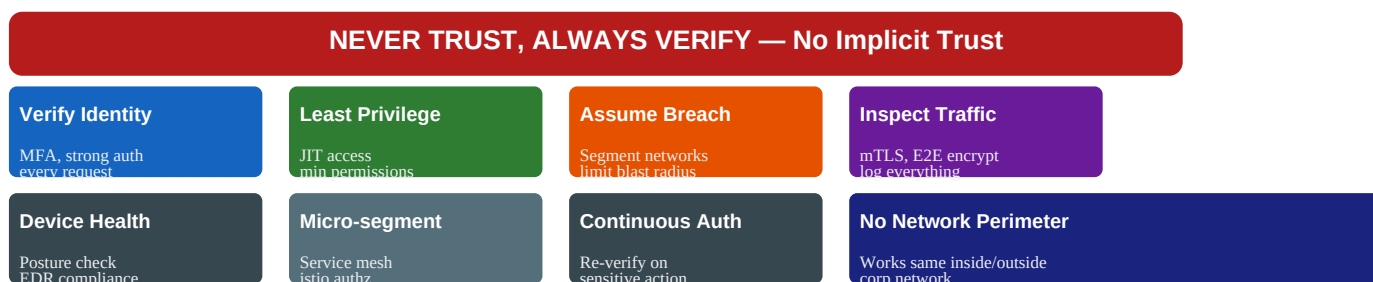
Rego policy is human-readable, version-controlled, testable. Deploy alongside app or as sidecar.

Input: {"user":"alice","action":"read","resource":"orders/123"}. Output: {"allow":true}.

```
# OPA Rego policy example – order access control
package orders.authz
import future.keywords.if
import future.keywords.in
default allow = false
# Admin can do anything
allow if { "admin" in input.user.roles }
# User can read their own orders
allow if { input.action == "read"
input.resource.owner_id == input.user.id }
# Manager can approve orders within their limit
allow if { input.action == "approve"
"manager" in input.user.roles
input.resource.amount <= input.user.approval_limit
time.hour(time.now_ns()) >= 9
time.hour(time.now_ns()) < 17 }

# Java: call OPA via REST API
// POST http://opa:8181/v1/data/orders/authz/allow
// {"input":{"user":{"id":"alice","roles":["manager"],"approval_limit":5000}, //
"action":"approve","resource":{"owner_id":"alice","amount":3000}}}
// Response: {"result":true}
```

Zero Trust Architecture



Zero Trust Principle	Traditional	Zero Trust
Network Trust	Trust internal network, block external	No network is trusted — verify every request
Authentication	Login once, trust session	Continuous verification, re-auth for sensitive ops
Service-to-Service	Trust via network segment (VLAN)	mTLS between every service, verify cert identity
Lateral Movement	Once inside, can reach anything	Micro-segmentation, least-privilege network policies

Device Trust	Trust domain-joined devices	Device posture check (EDR, patch level, cert)
Logging	Perimeter logs only	Log every request, anomaly detection, SIEM

Service Mesh & Istio Authorization

```
# Istio AuthorizationPolicy – zero-trust service-to-service apiVersion:
security.istio.io/v1beta1 kind: AuthorizationPolicy metadata: name: orders-service-policy
namespace: production spec: selector: matchLabels: app: orders-service action: ALLOW rules: -
from: - source: principals: # mTLS identity from SPIFFE/X.509 cert -
"cluster.local/ns/production/sa/api-gateway" -
"cluster.local/ns/production/sa/payment-service" to: - operation: methods: ["GET", "POST"]
paths: ["/api/orders/*"] --- # PeerAuthentication – enforce mTLS for all services apiVersion:
security.istio.io/v1beta1 kind: PeerAuthentication metadata: name: default namespace:
production spec: mTLS: mode: STRICT # Reject all non-mTLS traffic
```

Just-in-Time (JIT) Access & Privileged Access

Zero Trust applies to privileged access too. Instead of standing admin access, JIT access grants elevated permissions for a specific time window and purpose. AWS: IAM role assumption with session duration limit + CloudTrail audit. HashiCorp Vault: dynamic credentials generated on-demand, short TTL. PAM solutions: CyberArk, BeyondTrust.

50+ Interview Questions & Answers

Q1. What is RBAC?

Role-Based Access Control — a model where permissions are assigned to roles, and users are assigned to roles. Users get permissions indirectly through their roles. NIST RBAC standard defines 4 levels: flat, hierarchical, constrained, symmetric.

Q2. What is ABAC?

Attribute-Based Access Control — authorization decisions based on attributes of the subject (user), resource, action, and environment. Policy: PERMIT if subject.dept == resource.dept AND time.hour BETWEEN 9 AND 17. More flexible than RBAC.

Q3. What is the difference between RBAC and ABAC?

RBAC: user has a role, role has permissions — simple, coarse-grained. ABAC: policies combine multiple attributes — flexible, fine-grained, context-aware. Many systems combine both: RBAC for coarse access, ABAC for fine-grained data decisions.

Q4. What is ReBAC (Relationship-Based Access Control)?

Authorization based on relationships between entities. Google Zanzibar (used in Google Drive): 'user X has viewer relationship with document Y'. Enables natural permission models (shared documents, organization hierarchies). OpenFGA is an open-source Zanzibar implementation.

Q5. What is the principle of least privilege?

Grant only the minimum permissions required for a function. User can only access their own data. Service account can only read specific tables. Admin access requires explicit elevation. Reduces blast radius of compromise.

Q6. What is separation of duties?

No single person should have all permissions needed to complete a sensitive task. Example: the person who initiates a payment cannot also approve it. Requires at least two people — prevents insider fraud. Implemented as conflicting role constraints in RBAC.

Q7. What is Zero Trust?

A security model based on: 1) Never trust, always verify. 2) Assume breach. 3) Verify explicitly. Eliminates implicit trust based on network location (inside vs outside firewall). Every request is authenticated, authorized, and encrypted.

Q8. What are the core principles of Zero Trust?

1) Strong identity verification (MFA, device cert). 2) Least privilege access. 3) Micro-segmentation (limit lateral movement). 4) Assume breach mindset. 5) Continuous monitoring and validation. 6) Encrypt everything (mTLS between services).

Q9. What is micro-segmentation?

Dividing the network/application into small segments with independent access controls. Even if an attacker compromises one service, they cannot reach others without explicit authorization. Implemented via Kubernetes NetworkPolicy, Istio AuthorizationPolicy, or cloud security groups.

Q10. What is mTLS in Zero Trust?

Mutual TLS provides cryptographic service identity in Zero Trust. Each service has a certificate (issued by cert-manager or service mesh CA). Services verify each other's identity via certificate validation. Istio automates this for all service-to-service calls in Kubernetes.

Q11. What is SPIFFE?

Secure Production Identity Framework for Everyone — an open standard for service identity in dynamic environments. SPIFFE ID: `spiffe://trust-domain/path`. Implemented by SPIRE (runtime). Istio uses SPIFFE identities for service-to-service mTLS.

Q12. What is Open Policy Agent (OPA)?

An open-source, general-purpose policy engine. Policies written in Rego (declarative language). Input: JSON document describing the request context. Output: policy decision (allow/deny + additional data). Can be deployed as: sidecar, standalone service, or embedded library.

Q13. What is Rego?

The policy language for OPA. Declarative, logic-based. Policies are collections of rules. `rule := expression`. Rules are evaluated against input document. Human-readable and testable with `opa test`. Separates policy logic from application code.

Q14. What is the PEP/PDP/PAP/PIP pattern?

PEP (Policy Enforcement Point): enforces decisions in application. PDP (Policy Decision Point): evaluates policies (OPA). PAP (Policy Administration Point): where policies are written/managed. PIP (Policy Information Point): provides attribute data to PDP.

Q15. What is XACML?

eXtensible Access Control Markup Language — an XML-based standard for ABAC. Defines policy, request, response format. Complex but comprehensive. Used in enterprise and government. OPA/Rego is simpler and more widely adopted in cloud-native environments.

Q16. What is the difference between authentication and authorization?

Authentication: verifying who the user is (identity). Authorization: determining what the authenticated user is allowed to do (permissions). Authentication must happen before authorization. They are complementary —

both are necessary.

Q17. What is @Secured annotation?

Spring Security annotation for role-based method security: `@Secured('ROLE_ADMIN')`. Simpler than `@PreAuthorize` but less flexible (no SpEL). Requires `@EnableMethodSecurity(securedEnabled=true)`. Use `@PreAuthorize` for new code (more expressive).

Q18. What is @RolesAllowed?

JSR-250 annotation: `@RolesAllowed({'ROLE_ADMIN', 'ROLE_MANAGER'})`. Works with Spring Security when `jsr250Enabled=true` in `@EnableMethodSecurity`. More portable than `@Secured` (standard Java EE). Same functionality, different annotation source.

Q19. What is method security vs URL security?

URL security: coarse-grained — `requestMatchers('/admin/**').hasRole('ADMIN')`. Method security: fine-grained — `@PreAuthorize` checks at the service layer. Use both: URL security as first line of defense, method security for precise business rules.

Q20. What is SpEL in Spring Security?

Spring Expression Language used in `@PreAuthorize`. Built-in: `hasRole()`, `hasAuthority()`, `isAuthenticated()`, `#paramName`, `returnObject`, `authentication`, `principal`. Custom: `@beanName.method(#param)`. Evaluated at runtime against the security expression context.

Q21. What is the authentication object in SpEL?

`authentication` refers to the current `Authentication` object in the `SecurityContext`. `authentication.principal` — the `UserDetails` or `OAuth2User`. `authentication.name` — the username. `authentication.authorities` — collection of `GrantedAuthority`. `authentication.credentials` — password (usually cleared).

Q22. How do you implement row-level security in Spring?

At service layer: `@PreAuthorize` or `@PostFilter`. At JPA level: use `@Filter` or `QueryRewriter` to add `WHERE` clauses automatically. Example: `@Query('SELECT o FROM Order o WHERE o.userId = :#{authentication.principal.id}')`. Or JPA entity filtering with `@EnableJpaRepositories`.

Q23. What is Just-in-Time (JIT) access?

Granting elevated permissions only when needed, for a limited time window, for a specific task. Replaces standing admin privileges. AWS: assume an IAM role with session duration 1 hour. HashiCorp Vault: dynamic credentials with TTL. Reduces attack surface of privileged accounts.

Q24. What is Privileged Access Management (PAM)?

Tools and processes for managing, monitoring, and auditing privileged accounts (admins, service accounts). CyberArk, BeyondTrust, HashiCorp Vault. Features: password vaulting, session recording, access workflow, MFA for privileged access.

Q25. What is the difference between identity and access management (IAM) and PAM?

IAM: manages all user identities and access (both regular and privileged). PAM: subset focused specifically on privileged/admin accounts. PAM adds: session monitoring, privileged password vaulting, approval workflows for elevated access.

Q26. What is device posture in Zero Trust?

Verifying the security state of the device attempting to access a resource. Checks: OS version, patch level, endpoint detection & response (EDR) running, disk encryption enabled, certificate valid. If device fails posture check, deny access or require additional auth.

Q27. What is continuous authentication?

Periodically re-verifying user identity throughout a session, not just at login. Context-based: re-auth if: IP changes, device changes, unusual access pattern, long idle time. Sensitive operations (password change, payment) always require fresh authentication.

Q28. What is a service account?

A non-human identity used by applications and services. Should not be shared between services. Use separate accounts per service. Rotate credentials regularly. Apply least privilege. In Kubernetes: use ServiceAccount + RBAC. AWS: use IAM roles for EC2/ECS.

Q29. What is Kubernetes RBAC?

Kubernetes native role-based access control. Resources: Role/ClusterRole (define permissions), RoleBinding/ClusterRoleBinding (bind roles to subjects). Subjects: users, groups, ServiceAccounts. Used to restrict what services and users can do in the cluster.

Q30. What is OIDC and RBAC in Kubernetes?

Kubernetes can authenticate users via OIDC tokens. Extract user identity and group claims from JWT. Map to Kubernetes RBAC subjects via RoleBinding. Example: Azure AD group 'k8s-admin' → ClusterRoleBinding to cluster-admin role.

Q31. What is AWS IAM?

Identity and Access Management for AWS. Principals: users, groups, roles. Policies: JSON documents with Allow/Deny statements, actions, resources, conditions. Evaluation: explicit deny > explicit allow > implicit deny. Roles: assumed by services, federated users, cross-account.

Q32. What is the difference between IAM role and IAM user?

IAM user: permanent credential (access key + secret). IAM role: temporary, assumed by entities. Roles are preferred: temporary credentials rotate automatically, can be assumed by AWS services (EC2, Lambda), support cross-account and federation. Avoid long-lived user keys.

Q33. What is permission boundary in AWS IAM?

An advanced IAM feature that sets the maximum permissions a principal can have, even if their policies grant more. Used to delegate permission management safely: allow teams to create IAM roles but only within the boundary limits.

Q34. What is AWS IAM conditions?

Conditions in IAM policies restrict when the policy applies. Examples: aws:RequestedRegion, aws:SourceIp, aws:MultiFactorAuthPresent, s3:prefix. MFA condition: allow sensitive actions only when MFA was used. IP condition: allow only from corporate IP range.

Q35. What is the IAM condition key aws:PrincipalOrgID?

Restricts access to requests from principals within an AWS Organization. Use in resource-based policies to allow any account in your organization to access a resource — without listing each account ID explicitly.

Q36. What is OPA Gatekeeper?

OPA deployed in Kubernetes as an admission controller. Validates Kubernetes resource definitions against Rego policies. Examples: require labels on all resources, disallow privileged containers, enforce naming conventions. Policy-as-code for Kubernetes governance.

Q37. What is the difference between admission control and runtime authorization in K8s?

Admission control (OPA Gatekeeper): validates resources at creation time — prevents misconfigured pods.
Runtime authorization (Istio AuthorizationPolicy): controls east-west traffic at runtime — which services can talk to which. Both needed for defense in depth.

Q38. What is Kyverno?

An alternative to OPA Gatekeeper for Kubernetes policy management. Policies written in YAML (not Rego — no custom language). Can validate, mutate, and generate Kubernetes resources. Easier adoption than OPA for teams already familiar with Kubernetes YAML.

Q39. What is the principle of defense in depth applied to authorization?

Multiple authorization layers: API gateway (coarse), Spring Security filter (URL-level), method security (service-level), row-level in DB. If one layer is bypassed, others still protect. Never rely on a single authorization check.

Q40. What is an access token vs session token in Zero Trust?

Access token (JWT): short-lived, cryptographically verified, carries claims. Session token (cookie): validated against server-side session store. Zero Trust prefers tokens: stateless, carry identity/claims, can be verified without network call (JWT). Sessions require centralized storage.

Q41. What is Google Zanzibar?

Google's global authorization system powering Google Drive, YouTube, Maps permissions. Based on ReBAC (relationship-based). Tuples: user:alice has viewer relationship on document:doc1. Supports hierarchical permissions and group membership. Open source implementation: OpenFGA.

Q42. What is OpenFGA?

Open-source implementation of Google Zanzibar authorization model. CNCF project. Define relationships (owner, editor, viewer) and authorization model. SDK for Java, Go, Node.js. Used by Auth0, Okta. Suitable for: multi-tenant, document sharing, hierarchical permissions.

Q43. What is fine-grained authorization?

Authorization at the data/field level, not just endpoint level. Examples: user can read their own orders but not others', user can see basic profile but not SSN, manager can approve orders under their limit. Requires ABAC or ReBAC, not just RBAC.

Q44. How do you implement multi-tenancy authorization?

Tenant isolation: add tenantId to all entities, filter all queries by tenantId of current user. Service layer: verify tenantId matches. DB: use row-level security (PostgreSQL RLS) or schema-per-tenant. Spring: ThreadLocal TenantContext populated at request start.

Q45. What is Attribute Release Policy in OIDC/SAML?

IdP-side policy controlling which attributes are released to which SP. Example: IdP only sends email to low-trust apps, sends full profile only to verified apps. Prevents over-sharing of user data. Configurable in Keycloak, Auth0, Okta.

Q46. What is role explosion problem in RBAC?

As systems grow, the number of roles grows combinatorially (location × department × function × level). Hundreds of roles become unmanageable. Solution: use hierarchical RBAC, introduce ABAC for fine-grained decisions, or ReBAC for relationship-based logic.

Q47. What is the Istio PeerAuthentication vs RequestAuthentication?

PeerAuthentication: controls mTLS between services (STRICT = require mTLS, PERMISSIVE = allow both). RequestAuthentication: validates JWT tokens from end users (defines JWT issuers and JWKS). Both are

needed in Zero Trust: mTLS for service identity, JWT for user identity.

Q48. How do you audit authorization decisions?

Log: who (user/service), what (action), which resource, when, decision (allow/deny), policy that matched. Store in immutable audit log (CloudTrail, Vault audit, custom audit table). Alert on: repeated denials (probing), privilege escalation attempts, unusual patterns.

Q49. What is dynamic authorization vs static authorization?

Static: permission assigned at user creation time, doesn't change without admin action (traditional RBAC).
Dynamic: authorization decisions incorporate real-time context (time, location, device posture, risk score) — ABAC/Zero Trust principle. Most mature systems combine both.